API Gateway — JWT verification fix & code review

ma.ensa.apigateway.auth.util | Spring Boot 4.1.1, Java 24, MapStruct 1.6.3 | 7 November 2026

1. What I changed (exactly)

Three files touched, nothing else. The hardcoded client secret, the `master` realm and the missing claim validation were left exactly as they were, as you asked.

1.1 Jwt.java — added the signed input, renamed the signature component

BEFORE

```
public record Jwt(
        JwtHeader header,
        JwtPayload payload,
        byte[] Signature
) {}
```

AFTER

```
public record Jwt(
        JwtHeader header,
        JwtPayload payload,
        byte[] signature,
        // the exact "base64url(header).base64url(payload)" text as received,
        // it is what the RSA signature was computed over and cannot be rebuilt
        // from the decoded header/payload records.
        String signedInput
) {}
```

Why: the record threw away the only bytes that can be verified. Renaming `Signature` to `signature` also removes the visual clash with `java.security.Signature`, which is used in the same method.

1.2 JwtAdapter.java — one line

```
- return new Jwt(jwtHeader, jwtPayload, signature);
+ return new Jwt(jwtHeader, jwtPayload, signature, parts[0] + "." + parts[1]);
```

1.3 JwtUtil.java — dropped the mapper, fixed the verified bytes

BEFORE

```
private final TokenMapper mapper;

public JwtUtil(RestClient restClient, AuthProviderProperties properties, TokenMapper mapper) {
    ...
    this.mapper = mapper;
}

public Jwt getJwt() { ... return mapper.toJwt(tokenDto); }

public boolean verifyToken(Jwt jwt, Jwks jwks) {
    Jwk jwk = getJwk(jwt.header().kid(), jwks);
    byte[] signature = Base64.getUrlDecoder().decode(jwt.Signature());
    byte[] data = (jwt.header() + "." + jwt.payload()).getBytes();
    ...
```

AFTER

```
private final JwtAdapter jwtAdapter;

public JwtUtil(RestClient restClient, AuthProviderProperties properties, JwtAdapter jwtAdapter) {
    ...
    this.jwtAdapter = jwtAdapter;
}

public Jwt getJwt() { ... return jwtAdapter.buildJwt(tokenDto); }

public boolean verifyToken(Jwt jwt, Jwks jwks) {
    Jwk jwk = getJwk(jwt.header().kid(), jwks);
    byte[] signature = jwt.signature();
    byte[] data = jwt.signedInput().getBytes(StandardCharsets.US_ASCII);
    ...
```

Plus one import: `java.nio.charset.StandardCharsets`.

2. Why it was failing — three stacked bugs

#	Bug	Symptom
1	**MapStruct never called your JwtAdapter.** `uses = {JwtAdapter.class}` supplies *property-level* converters; it does not implement the root `toJwt` method. So MapStruct mapped property-by-property, and `TokenDto(accessToken, exp, tokentype)` has no name in common with `Jwt(header, payload, Signature)` — the generated `TokenMapperImpl` was literally `new Jwt(null, null, null)`.	The NPE you hit: `jwt.header()` is null at JwtUtil.java:80.
2	**The wrong bytes were verified.** `(jwt.header() + "." + jwt.payload()).getBytes()` is the record's `toString()` — `JwtHeader[alg=RS256, ...]`. RSA verification must run over the original base64url text, byte for byte.	Would have returned `false` for every valid token.
3	**Signature decoded twice.** `JwtAdapter` already base64url-decoded `parts[2]`; `verifyToken` decoded the result again.	Garbage bytes, or `IllegalArgumentException`.

Note: bug 1 should have appeared as an *Unmapped target property* javac warning at build time. Worth reading those — or promoting them to errors with `-Amapstruct.unmappedTargetPolicy=ERROR`.

Verified

`mvn compile` clean, the app booted against Keycloak on localhost:8080, and the CommandLineRunner printed **true**. I had to start it on a free port — something was already holding 8888.

`TokenMapper.java` is now unused. It still compiles and still produces an all-null Jwt, so delete it when you get the chance — I left it in place to keep the change minimal.

3. Rating of the handwritten code

Area	Score	Comment
Design & decomposition	**8 / 10**	Genuinely good instinct. Records for Jwt / JwtHeader / JwtPayload / Jwk / Jwks, a dedicated adapter for decoding, the RestClient built once in a `@Configuration`, the base URL in `@ConfigurationProperties`. That is how a gateway auth layer should be laid out.
Statelessness	**9 / 10**	Your own commented note in JwtUtil — that a mutable `access_token` field on a singleton bean is a concurrency bug and the value belongs in a parameter — is exactly right, and is the most mature thing in the file. Keep that reflex.
Cryptographic correctness	**3 / 10**	The RSA/JWK plumbing is right: unsigned BigInteger, RSAPublicKeySpec, SHA256withRSA, kid lookup in the JWKS. But *what* got signed was wrong, and a verifier that cannot verify is worth nothing. This is where the learning is.
Framework fluency	**5 / 10**	MapStruct was reached for where a direct call was simpler and correct — a mapper is for field-to-field copying, not for parsing. Also Gson and Jackson are both in play for the same objects; pick one.

Error handling	**4 / 10**	Four identical catch blocks each wrapping in RuntimeException; `parts[0..2]` indexed with no length check; `tokenDto` can be null before `.accessToken()` is called.
Style & hygiene	**5 / 10**	Capitalised record component `Signature`, `snake_case` locals, inconsistent spacing after commas, large commented-out blocks, an unused `properties` field. Cosmetic — but it is what a reviewer notices first.
Security posture	**2 / 10**	Secret in source; the signature is checked but `alg`, `exp`, `iss` and `aud` are not. Untouched per your instruction — listed here only because this is a DevSecOps project.

Overall: about 5.5 / 10 — a better mark than it sounds. Hand-rolling JWS verification off the JWKS endpoint, with no library, is a real exercise, and the architecture around it is sound. What let it down is a category of mistake rather than sloppiness: treating a parsed object as if it still carried its wire representation. Fix that habit and this is an 8 fairly quickly.

4. What to focus on next

The concept to internalise first

A signature belongs to bytes, not to objects. The moment you base64-decode and deserialise a JWT, the signed artifact is gone: JSON key order, whitespace, and every claim your record does not declare are unrecoverable. In any signed-payload format — JWS, webhook HMACs such as Stripe or GitHub, SAML, signed git objects — always keep the raw bytes alongside the parsed view. That single idea covers most signature-verification bugs you will ever write.

Then, in order

1. **Validate the claims, not just the maths**
   A signature proves authorship and nothing else. A correctly signed but expired token passes today. Add, in this order: `alg` must equal `RS256` — never let the token's own header choose the algorithm, that is the classic algorithm-confusion and `alg: none` attack; then `exp` / `nbf` against `Instant.now()` with a small clock skew; then `iss` and `aud`. Type `exp` and `iat` as `long` while you are in there — they are JSON numbers, not strings.

2. **Write the tests you could not have passed**
   Three cases: a valid token returns true; a token with one flipped payload byte returns false; an expired token is rejected. The second is the test that would have caught every bug in this file. Verification code with no negative test is untested code.

3. **Cache the JWKS**
   You fetch `/certs` on every verification. In a gateway that is a network round-trip per request and a hard dependency on Keycloak being up. Cache by `kid`, and refetch only when an unknown `kid` shows up — that is what key rotation looks like from here.

4. **Then delete most of this code**
   Having written it by hand, which was the right way to learn it, move the gateway to `spring-boot-starter-oauth2-resource-server`. Point `spring.security.oauth2.resourceserver.jwt.issuer-uri` at the realm and Nimbus handles signature, JWKS caching, rotation, clock skew and claim validation, all audited. Knowing exactly what it is doing for you is the whole point of the exercise.

5. **Housekeeping**
   Delete `TokenMapper` and the commented-out blocks; drop either Gson or Jackson; remove the unused `properties` field; collapse the four catch blocks into `catch (GeneralSecurityException e)`; guard `parts.length == 3`.

Files changed: `auth/util/Jwt.java`, `auth/util/JwtAdapter.java`, `auth/util/JwtUtil.java`. Secret, realm and claim validation deliberately untouched.